TN PRICE MONITOR

Real-Time Commodity Price Tracking Platform

Technical Documentation

High Level Design | Data Flow | Tech Stack

Project	TN Price Monitor
Version	1.0.0 — Production
Live URL	https://tn-price-tracker-9ekp.vercel.app
GitHub	github.com/deepakharish17/TN_PRICE_TRACKER
Author	Deepak Harish — harishdeepak35@gmail.com
Date	March 2026

1. Project Overview

TN Price Monitor is a full-stack, community-driven web platform that enables farmers, buyers, traders, and homemakers across Tamil Nadu to submit, track, and analyse live commodity prices across all 25 districts in real time. The platform operates on a crowdsourced, admin-verified model — ensuring data accuracy while empowering communities to contribute.

The platform serves two primary personas:

- Users (Farmers, Buyers, Public) — Submit prices, browse market rates, track submissions, compare districts, view analytics and leaderboard
- Admins (Moderators) — Review and approve submissions, manage users, post announcements, export data, monitor platform activity

Core Problem Solved

Farmers in Tamil Nadu travel long distances to markets without knowing current commodity prices, leading to poor decisions, financial losses, and market inefficiency. TN Price Monitor solves this by giving every citizen a phone-friendly tool to check live prices and contribute data from their local market — creating a continuously updated, crowd-verified price database for all of Tamil Nadu.

2. High Level Design (HLD)

2.1 System Architecture

The system follows a classic 3-tier architecture: Presentation Layer (React frontend), Application Layer (Node/Express REST API), and Data Layer (MongoDB Atlas). The frontend is deployed on Vercel (CDN-backed, global edge), the backend on Render (Node.js server), and the database on MongoDB Atlas (cloud-hosted, M0 free tier).

SYSTEM ARCHITECTURE OVERVIEW

[Browser / Mobile]

↕ HTTPS

[Vercel CDN — React 18 + Vite Frontend]

↕ REST API (HTTPS / JSON)

[Render — Node.js + Express Backend]

↕ MongoDB Wire Protocol (TLS)

[MongoDB Atlas — Cloud Database]

↕ HTTP Ping (every 5 min)

[UptimeRobot — 24/7 Keep-Alive Monitor]

2.2 Frontend Architecture

The frontend is a Single Page Application (SPA) built with React 18 and Vite. It uses React Router v6 for client-side routing with ProtectedRoute wrappers enforcing role-based access. State is managed locally via React hooks. Axios handles all API communication with a centralized interceptor injecting JWT tokens.

Layer	Details
Routing	React Router v6 — /dashboard, /admin, /market-prices, /analytics, /compare, /leaderboard, /profile, /add-price, /bookmarks, /export, /settings, /audit-log, /activity-heatmap, /announcements, /weather, /price-history, /pending-review, /admin-users, /admin-dashboard, /notifications, /my-submissions
Auth Guard	ProtectedRoute.jsx — checks localStorage token + role; redirects to /login if unauthorized
API Layer	src/api/axios.js — Axios instance with baseURL from VITE_API_URL env variable + Bearer token interceptor
Theming	CSS variables in index.css — dark/light mode toggle stored in localStorage; --primary, --bg, --card-bg, --text-primary
Charts	Recharts — LineChart (price trends), BarChart (district comparison), PieChart (commodity distribution)
Responsive	3 breakpoints: Desktop 1024px+, Tablet 768px, Mobile 480px — bottom nav bar + hamburger drawer on mobile

2.3 Backend Architecture

The backend is a RESTful API built with Node.js and Express.js. It follows an MVC-like structure: routes define endpoints, middleware handles authentication and authorization, and models define Mongoose schemas. All routes are prefixed with /api/.

Route Group	Base Path	Key Endpoints
Auth	/api/auth	POST /register, POST /login
Prices	/api/price	GET / (all approved), POST / (submit), GET /my, GET /history, GET /compare, GET /export
Admin	/api/admin	GET /pending, PATCH /approve/:id, PATCH /reject/:id, GET /users, PATCH /suspend/:id, GET /audit-log, GET /settings, GET /activity
Notifications	/api/notifications	GET / (user notifs), PATCH /read/:id, PATCH /read-all
Announcements	/api/announcements	GET / (all), POST / (admin only), DELETE /:id (admin only)

2.4 Database Design

MongoDB Atlas with Mongoose ODM. The database is named 'test' (Atlas default). Six collections are used:

Collection	Key Fields	Purpose
users	name, email, password (bcrypt), role, suspended, suspend Reason	Auth + user management
prices	commodity, marketName, district, price, status, userId (ref), rejectionReason	Core commodity price data — pending / approved / rejected
notifications	userId (ref), message, type, read	Per-user notification feed
announcements	title, body, type, postedBy (ref)	Platform-wide broadcasts by admins
auditlogs	adminId, adminName, action, targetName, details	Immutable log of every admin action
settings	key, value (Mixed)	Configurable districts and commodities list

3. Data Flow

3.1 User Authentication Flow

Step	Actor	Action
1	**User (Browser)**	Enters email + password on /login page, clicks Sign In
2	**Frontend (axios)**	POST /api/auth/login with { email, password }
3	**Backend (authRoutes)**	Finds user by email in MongoDB → compares password with bcrypt.compare()
4	**Backend**	If valid: signs JWT with user._id + role (expires 7d) → returns { token, user }
5	**Frontend**	Stores token in localStorage → decodes role → redirects to /dashboard or /admin-dashboard
6	**All future requests**	Axios interceptor attaches Authorization: Bearer <token> to every request header
7	**Backend middleware**	auth.js extracts + verifies JWT → attaches req.user → passes to route handler

3.2 Price Submission Flow (User → Admin → Live)

Step	Actor	Action
1	**User**	Fills Add Price form — selects commodity, district, market, enters price
2	**Frontend**	POST /api/price with JWT token in header
3	**Backend**	Validates data → saves to 'prices' collection with status: 'pending'
4	**Backend**	Creates notification for the submitting user — 'Submission received'
5	**Admin**	Logs into admin dashboard → sees new entry in Pending Review queue
6	**Admin**	Reviews price — clicks Approve / Edit+Approve / Reject with reason
7	**Backend**	Updates price document status to 'approved' or 'rejected'
8	**Backend**	Creates notification for the user — 'Your submission was approved/rejected'
9	**Backend**	If approved: writes to AuditLog collection with admin details
10	**All Users**	Approved price is now visible on Market Prices board in real time

3.3 Analytics Data Flow

When a user opens the Analytics page, the frontend calls GET /api/price with optional filters (commodity, district, date range). The backend queries MongoDB using Mongoose aggregation pipelines to compute average, min, and max prices grouped by date. Results are returned as JSON arrays which Recharts renders as interactive line charts, bar charts, and pie charts in the browser.

3.4 Role-Based Access Control (RBAC) Flow

Resource	User Role	Admin Role
Market Prices (read)	☑ Allowed	☑ Allowed
Submit Price	☑ Allowed	☑ Allowed
View Own Submissions	☑ Allowed	☑ Allowed
Approve / Reject Prices	✕ Blocked	☑ Allowed
Manage Users	✕ Blocked	☑ Allowed
Post Announcements	✕ Blocked	☑ Allowed
View Audit Log	✕ Blocked	☑ Allowed
Export CSV	✕ Blocked	☑ Allowed
Edit Settings	✕ Blocked	☑ Allowed

4. Tech Stack

4.1 Frontend

Technology	Version	Usage
React	18.x	Core UI library — component-based SPA architecture
Vite	5.x	Build tool — fast HMR in dev, optimized bundle for production
React Router	v6	Client-side routing — nested routes, protected routes, navigation
Axios	1.x	HTTP client — centralized API calls with JWT interceptor
Recharts	2.x	Charts — LineChart, BarChart, PieChart, AreaChart for analytics
Pure CSS	—	Custom styling — CSS variables, dark/light mode, responsive grid, animations
Open-Meteo API	Free	External weather API — real-time temperature and rainfall data by district

4.2 Backend

Technology	Version	Usage
Node.js	18.x	JavaScript runtime — event-driven, non-blocking I/O
Express.js	4.x	Web framework — REST API routing, middleware pipeline
Mongoose	7.x	MongoDB ODM — schema definition, validation, query building
bcryptjs	2.x	Password hashing — salted bcrypt rounds for secure storage
jsonwebtoken	9.x	JWT generation and verification — stateless auth tokens (7d expiry)
env	16.x	Environment variable management — MONGO_URI, JWT_SECRET, PORT
cors	2.x	Cross-origin request handling — allows Vercel frontend to call Render backend

4.3 Database

Technology	Tier	Usage
MongoDB Atlas	M0 Free	Cloud-hosted NoSQL database — flexible document model for price data
Mongoose ODM	—	Schema validation, middleware hooks, population (userId ref joins)

4.4 Deployment & DevOps

Service	Platform	Configuration
Frontend Hosting	Vercel	Auto-deploy from GitHub main branch — build: npm run build, output: dist, env: VITE_API_URL
Backend Hosting	Render	Web Service — root: backend, build: npm install, start: node server.js, env: MONGO_URI + JWT_SECRET
Database	MongoDB Atlas	M0 free cluster — IP whitelist 0.0.0.0/0, database: test, user: TN-PRICE-TRACKER
Uptime Monitor	UptimeRobot	HTTP monitor pings Render backend every 5 minutes — prevents free tier sleep
Version Control	GitHub	Main branch — Vercel and Render both auto-redeploy on every git push

4.5 Security Implementation

Security Layer	Implementation
Password Storage	bcryptjs with 10 salt rounds — passwords never stored in plain text
Authentication	JWT (jsonwebtoken) — signed with secret key, 7-day expiry, Bearer token scheme
Authorization	Role-based middleware — isAuth.js checks token, isAdmin.js checks role field
CORS	Express CORS middleware — only allows requests from the Vercel frontend domain
Environment Secrets	All secrets (MONGO_URI, JWT_SECRET) in environment variables — never in code
Input Validation	Mongoose schema validation — required fields, type checking on all price submissions

5. Platform Coverage & Scale

Metric	Value
Districts Covered	All 25 Districts of Tamil Nadu
Commodities Tracked	16 (Vegetables, Grains, Pulses, Oils, Dairy)
Markets per District	5 local market names each
Seed Data — Prices	~2,000+ approved entries
Price History Range	30-day trend data
User Roles	2 (user, admin)
Total Pages / Screens	22 pages
Mobile Support	Full — bottom nav, drawer sidebar, responsive grids
Hosting Cost	₹0 — fully free tier stack

6. Social Impact

TN Price Monitor directly addresses market information asymmetry — one of the biggest challenges facing small-scale farmers in Tamil Nadu. By democratizing access to real-time commodity price data, the platform delivers tangible benefits across multiple segments of society:

Beneficiary	Benefit
Farmers	Can check today's prices before travelling to market — save fuel, time, and avoid selling at loss
Buyers & Traders	Compare prices across districts to find the best source for commodities
Homemakers	Plan monthly grocery budget with live and trend data for essential commodities
NGOs & Government	Monitor price spikes in real time — enabling faster intervention during shortages or floods
Students & Researchers	Access structured price trend data for Tamil Nadu agriculture studies